01 / EXECUTIVE BRIEF

# Decide before you build.

Salesforce order-integration review | Illustrative deliverable | 23 October 2026

## The business decision

A fictional distributor wants approved Salesforce orders sent to an external fulfillment API. The API may be unavailable for up to 30 minutes. Sales staff must keep working, and retries must not create duplicate fulfillment orders.

## Recommended direction

Use an asynchronous integration with durable request state, a stable idempotency key and a status/reconciliation process. Keep user approval separate from the external API response. Validate API behavior and platform constraints before selecting an implementation pattern.

## What this review provides

A decision record, a conceptual data flow, prioritized risks and an acceptance checklist. This is an independent demonstration of the deliverable format. No org was inspected, no integration was implemented, and no performance or security test was executed.

## Assumptions to verify

One Salesforce org and one external API; Order and OrderItem are licensed and suitable; target volume assumed at 200 orders/day with bursts of 20 approvals/minute; 15-minute delivery objective when the API is healthy. These are synthetic planning assumptions, not supported-capacity claims.

## Approval gate

Confirm volumes, API limits, identity model, data permissions, recovery objectives and operational ownership before implementation. A written blueprint does not authorize production deployment.

02 / DECISION RECORD

# Make the trade-offs visible.

ADR-001 | Status: proposed, pending discovery and validation

## Option A - synchronous call from approval

Advantage: an immediate response. Trade-off: user-facing work becomes coupled to API availability and timeout behavior. Choose only if the business requires immediate confirmation and the callout/transaction boundaries are proven suitable.

## Option B - asynchronous delivery (recommended)

Advantage: approval can finish while delivery waits or retries. Trade-off: eventual consistency, persistent state, alerting and reconciliation become explicit responsibilities. The correct Salesforce mechanism depends on volume, transactional constraints, delivery guarantees and existing middleware.

## Option C - managed integration platform

Advantage: may reuse existing monitoring, connectors and support ownership. Trade-off: licensing, operating cost and another runtime. Prefer evaluation when the company already uses middleware or has several integration endpoints. MuleSoft is an option to assess, not a mandatory purchase.

## Decision and boundaries

Recommend asynchronous delivery with one authoritative delivery state and an external idempotency contract. Do not select Flow, Queueable Apex, events or middleware solely from a diagram: verify callout limits, publishing/commit behavior, retry semantics, API constraints and support capability first.

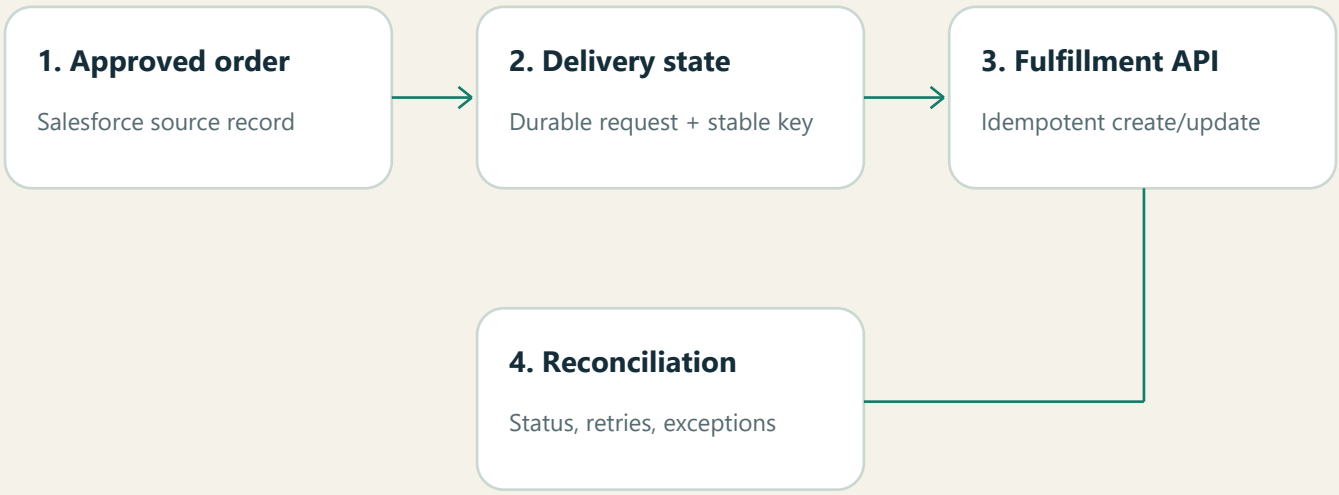
## Evidence required to close this decision

API contract and timeout/rate-limit behavior; duplicate-request handling; expected peak volumes; current approval automation; integration identity permissions; error ownership and retention requirements. Unknowns remain assumptions until evidence is supplied.

03 / TARGET DESIGN

# A controlled delivery path.

Conceptual pattern - implementation mechanism intentionally not yet selected




## Minimum data contract

Send approved fields only: external order key, approved line identifiers, quantities and permitted fulfillment fields. Keep customer personal data out of logs. Store the external reference, delivery state, attempt count and timestamps; do not log credentials or full sensitive payloads.

## Security and ownership

Use an approved integration identity, least privilege and managed credential storage. Separate who can approve, who can deliver and who can replay requests. Agree an owner for failures and a policy for retention and deletion. Validate network and authentication requirements in discovery.

## Failure behavior

Reuse the same key after an uncertain timeout; never create a fresh order key for a retry. Apply bounded retries/backoff for transient errors. Route validation failures and exhausted attempts to a visible exception queue with a controlled replay process.

04 / RISK REGISTER

# Prioritize what can go wrong.

Candidate risks derived from the fictional scenario - not observed defects

### P1 / Duplicate fulfillment after timeout

Impact: duplicate shipment or cost. Evidence: API idempotency contract and timeout simulation. Mitigation: stable business key, duplicate-safe endpoint behavior and reconciliation. Owner: integration lead; validate before release.

### P1 / Excessive data exposure

Impact: unauthorized customer or order data access. Evidence: field-level permissions, identity scope and log inspection. Mitigation: approved field allowlist, least privilege and redacted errors. Owner: security/platform owner.

### P2 / Silent delivery backlog

Impact: approved orders do not reach fulfillment in time. Evidence: queue age, failed-attempt visibility and alert test. Mitigation: accountable operations owner, monitored backlog and a documented runbook.

### P2 / Unproven capacity and retry behavior

Impact: bursts or API limits create repeated failures. Evidence: representative-volume tests and API rate-limit documentation. Mitigation: measured concurrency, bounded retry policy and volume-aware scheduling.

### P2 / Uncontrolled replay or schema change

Impact: wrong or inconsistent order content. Evidence: replay authorization, mapping versions and contract tests. Mitigation: explicit update policy, versioned mappings and approval for changed business semantics.

05 / DELIVERY ROADMAP

# Turn recommendations into action.

Suggested sequence; effort estimates require discovery and implementation scope

## Phase 1 - validate the design

Confirm the API contract, data classification, field mapping, peak volume and permitted identities. Agree the chosen delivery mechanism and record the alternatives. Exit: decision owner signs off the scope, risks and acceptance cases.

## Phase 2 - implement in a sandbox

Implement only the agreed process. Add stable request identity, visible state, bounded retries and actionable errors. Exercise success, timeout, duplicate request, validation failure, permission denial and API rate limiting. Exit: expected outcomes evidenced, unresolved risks documented.

## Phase 3 - prepare controlled release

Document configuration, support ownership, alert thresholds, replay permissions and rollback steps. Rehearse with sanitized data. Exit: business and platform owners approve deployment; production changes remain separately authorized.

## Acceptance examples

Duplicate-safe: replaying one approved order produces one target business order, according to the verified API contract.
Recoverable: a simulated outage produces visible pending/failed states and successful recovery without silent loss.
Controlled: an unauthorized user cannot trigger or replay delivery.
Observable: an owner can find aged requests and resolve an exception using the runbook.

## Deliverable limits

This sample is design advice, not an implementation, compliance certification, penetration test, exhaustive org audit or production-readiness guarantee. Licensing, platform limits and current vendor documentation must be checked for the actual environment.